Code walkthrough

instruction_memory.v

Instruction Memory

What it does	Stores the program (16 words x 12 bits)
Where it works	IF stage
Size	36 lines, explained in 5 small parts
Modules in this file	instruction_memory
Calls (uses)	nothing (built-in $readmemh only)
Called by	cpu_top.v - instance IMEM tb_cpu.v - task load_program writes CPU.IMEM.mem

The instruction memory holds the program. Given a 4-bit address it outputs the 12-bit instruction stored there. The program is loaded from a text file at the start of simulation.

Map of the file

Part	Lines	What this part is
1	1-15	Header comment
2	16-21	Parameter and ports
3	22-23	The storage
4	25-32	Loading the program
5	34-36	Reading

> How to read this document: each part below shows a few lines of the real file, with their line numbers, followed by a plain-language explanation. Read the parts in order to understand the whole file without scrolling through it in one go.

Part 1 · Header comment (lines 1-15)

instruction_memory.v lines 1-15

```
 1  // ====================================================================
 2  // FILE: instruction_memory.v
 3  // Stores the program: 16 words x 12 bits (an initialised ROM).
 4  //
 5  // Interface (as in the project plan):
 6  //   address[3:0], instruction[11:0]
 7  //
 8  // The program is read from a text file (default: program.mem).
 9  // Each line of the file is one instruction in hex, e.g. 645
10  //
11  // CONNECTIONS
12  //   Calls       : nothing (built-in $readmemh only)
13  //   Called by   : cpu_top.v - instance IMEM
14  //                 tb_cpu.v - task load_program writes CPU.IMEM.mem
15  // ====================================================================
```

- States the size (16 x 12), the interface and the file format (one hex word per line).
- The **CONNECTIONS** lines at the end of the header list which other files this file uses (Calls) and which files use it (Called by).

Part 2 · Parameter and ports (lines 16-21)

instruction_memory.v lines 16-21

```
16  module instruction_memory #(
17      parameter PROGRAM_FILE = "program.mem"   // file with the program
18  )(
19      input  [3:0]  address,        // from the PC
20      output [11:0] instruction     // instruction at that address
21  );
```

- `#( parameter PROGRAM_FILE = "program.mem" )` is a setting that can be changed from outside. The testbench uses it to choose which program to run.
- Input: `address` (from the PC). Output: the 12-bit `instruction`.

Part 3 · The storage (lines 22-23)

instruction_memory.v lines 22-23

```
22      reg [11:0] mem [0:15];        // 16 words, 12 bits each
23      integer i;
```

`reg [11:0] mem [0:15]` means 16 locations (0 to 15), each 12 bits wide. `integer i` is a loop counter used only during start-up.

Part 4 · Loading the program (lines 25-32)

instruction_memory.v lines 25-32

```
25      initial begin
26          // fill every word with C00 first (C = reserved opcode = NOP),
27          // so an empty location never contains unknown (X) values
28          for (i = 0; i < 16; i = i + 1)
29              mem[i] = 12'hC00;
30          // then load the program from the file
31          $readmemh(PROGRAM_FILE, mem);
32      end
```

- `initial` runs once at time 0.
- First every word is filled with `12'hC00`. Opcode C (1100) is reserved, so C00 is a NOP: unused locations are harmless and never contain unknown (X) values.
- `$readmemh(PROGRAM_FILE, mem)` then reads the hex file into the memory. In Vivado the same code also puts the program into the FPGA bitstream.

Part 5 · Reading (lines 34-36)

instruction_memory.v lines 34-36

```
34      // read: output the instruction at the given address
35      assign instruction = mem[address];
36  endmodule
```

`assign instruction = mem[address];` The output always shows the word at the current address. No clock is needed to read.

> **In short:** Address in → instruction out; contents come from a .mem file. **Example:** if line 3 of program.mem is 060, then address = 0010 gives instruction = 0000 0110 0000 (ADD R1, R2). A location not in the file gives C00 (NOP).